Neural Networks — CSE 589

Project Q&A: Fake News Detection with Deep Learning

30 Questions · 8 Topics · Covering Lectures 2–7

Q = question the examiner asks | **A** = expected answer | Section headers show lecture reference

1 — Project Overview

Q1. What is the goal of this project?

A: To automatically classify news articles as **Fake News** or **Factual News** using deep learning. A Feedforward Neural Network is built with TensorFlow/Keras, trained on TF-IDF features extracted from cleaned article text, and served through a FastAPI backend.

Q2. What dataset did you use and what does it contain?

A: 198 news articles with 4 original columns: **title, text, date, fake_or_factual**. The dataset is nearly balanced: 100 Factual and 98 Fake articles. After cleaning in Notebook 01, it was expanded to 12 columns including engineered features such as word_count, text_length, and title_upper_ratio.

Q3. Why is this a classification problem and not regression?

A: The output is a **discrete category** (Fake or Factual), not a continuous value. Binary classification with a Sigmoid output node is appropriate — it produces a probability in (0, 1) that is thresholded at 0.5 to assign a class label.

2 — Data Preprocessing *Lecture 2*

Q4. What data cleaning steps were applied to the text?

A: Lowercasing all text, removing URLs/emails, stripping HTML tags, removing punctuation and digits, removing 198 English stopwords, and filtering single-character tokens. Missing values and duplicate rows were also checked and eliminated.

Q5. What is TF-IDF and why was it used?

A: **TF-IDF (Term Frequency-Inverse Document Frequency)** converts text into a numerical vector. TF measures how often a word appears in a document; IDF reduces the weight of words common across all documents (like 'the'). The result is a 3401-dimensional vector per article that the Dense layers can process.

Q6. What is the curse of dimensionality and how was it addressed?

A: The curse of dimensionality refers to data becoming exponentially sparse as feature count grows. We addressed it by limiting TF-IDF to **max_features=5000** and **min_df=2** (drop words appearing in fewer than 2 documents), reducing vocabulary to 3401 terms.

3 — Neural Network Architecture *Lecture 3*

Q7. Describe the architecture of the main neural network.

A: Input(3401) → Dense(256, **ReLU**) → Dropout(0.5) → Dense(128, **ReLU**) → Dropout(0.5) → Dense(64, **ReLU**) → Dropout(0.3) → Dense(1, **Sigmoid**). Total trainable parameters: 912,129.

Q8. Why was ReLU chosen for hidden layers and Sigmoid for the output?

A: **ReLU** (max(0,x)) avoids the vanishing gradient problem — its derivative is 1 for positive inputs, not saturating like Sigmoid or Tanh. This makes deep networks train faster. **Sigmoid** squashes the output to (0, 1), representing the probability of Fake News, which is ideal for binary classification.

Q9. What is the vanishing gradient problem? Which activations cause it and how did you observe it?

A: Vanishing gradient occurs when gradients become near-zero during backpropagation in deep networks, stopping earlier layers from learning. **Sigmoid** (max derivative 0.25) and **Tanh** both saturate. In our activation comparison experiment: Sigmoid needed **66 epochs** to converge, Tanh needed **20 epochs**, while ReLU converged in only **23 epochs**.

Q10. How does forward propagation work in your network?

A: The input TF-IDF vector passes through each layer sequentially. Each Dense layer computes **z = Wx + b**, applies the activation f(z), and passes the result forward. The final Sigmoid layer outputs a probability p ∈ (0, 1) representing P(Fake).

Q11. How does backpropagation work?

A: Backpropagation uses the **chain rule** to compute the gradient of the loss with respect to each weight, working backwards from the output. The binary cross-entropy loss gradient flows back layer by layer — each weight is updated by: w = w - lr * (dL/dw). Keras/TensorFlow handles this automatically during model.fit().

4 — Training: Loss, Optimizers & Batching *Lecture 4*

Q12. What loss function was used and why?

A: **Binary Cross-Entropy**: -[y·log(y-hat) + (1-y)·log(1-y-hat)]. This is the standard loss for binary classification with a Sigmoid output. It heavily penalises confident wrong predictions and is derived from Categorical Cross-Entropy for the two-class case.

Q13. What optimizer was used? Compare it to SGD and RMSProp.

A: **Adam (Adaptive Moment Estimation)** combines momentum (moving average of gradients) with RMSProp (moving average of squared gradients), giving each weight its own adaptive learning rate. Our experiment confirmed: SGD = **60%** accuracy, RMSProp = **97.5%**, Adam = **97.5%**. SGD's fixed learning rate makes it unreliable on this task.

Q14. What training method (batch type) was used and why?

A: **Mini-batch gradient descent** with batch_size=16. This updates weights after every 16 samples — faster than full-batch (all 158 samples per update) and more stable than SGD (1 sample per update). Mini-batch is the standard choice in deep learning.

Q15. What is Early Stopping and why was it important here?

A: Early Stopping monitors **validation loss** and stops training when it fails to improve for patience=15 consecutive epochs, then restores the best weights. With only 198 samples the risk of overfitting is high. The model stopped at epoch 23 out of a maximum of 200.

5 — Regularization *Lecture 5*

Q16. What is Dropout and how does it prevent overfitting?

A: Dropout randomly zeros a fraction of neuron outputs each training step (50% in layers 1-2, 30% in layer 3). This forces the network to learn **redundant representations** — no single neuron can be relied upon. During inference all neurons are active. Our experiments showed Dropout converged in 22 epochs vs L2-only which needed 157 epochs.

Q17. What is L2 regularization and how does it differ from Dropout?

A: L2 regularization adds a penalty **lambda * sum(w^2)** to the loss, discouraging large weights and favouring simpler solutions. Unlike Dropout (which randomly disables neurons), L2 penalises weight magnitude every step. We used lambda=0.0001. In practice, **Dropout + L2 combined** often gives the most stable results.

Q18. What is the difference between overfitting and underfitting?

A: **Overfitting**: the model memorises training data — high training accuracy, low validation/test accuracy. Widening gap in loss curves. **Underfitting**: the model is too simple — low accuracy on both sets. We addressed overfitting with Dropout, Early Stopping, and L2 regularization; the chosen architecture has enough capacity to avoid underfitting.

6 — Convolutional Neural Networks *Lectures 5–7*

Q19. What is a CNN and how does it differ from a Feedforward NN?

A: A CNN uses **convolutional filters** (kernels) that slide over the input with shared weights, giving local receptive fields. A fully connected layer for a 400x500 image would need ~475 billion weights; a 3x3 convolutional filter needs only ~108. CNNs learn local patterns (edges, shapes) rather than global statistics.

Q20. How was a 1D CNN applied to text data?

A: Text is tokenised (words to integer IDs), padded to length 200, then an **Embedding** layer maps each word ID to a 64-dimensional dense vector. **Conv1D filters** of size 5 and 3 slide over this word sequence, detecting local n-gram patterns (e.g. 'not true', 'according to sources'). The architecture: Embed(5000,64) → Conv1D(128,5) → MaxPool(2) → Conv1D(64,3) → GlobalMaxPool → Dense(64) → Dense(1,Sigmoid).

Q21. What does MaxPooling do? Name both types used in Notebook 04.

A: MaxPooling downsamples feature maps by keeping only the **maximum value** in each pooling window, reducing spatial size and retaining the strongest features. **MaxPooling1D(2)** halves the sequence length (196 → 98 tokens). **GlobalMaxPooling1D** takes the single maximum value across the entire remaining sequence, collapsing it to a fixed-length vector regardless of length.

Q22. Calculate the output size after Conv1D(128, kernel=5) on input of length 200 (no padding).

A: Using the formula O = (W - F + 2P) / S + 1: O = (200 - 5 + 0) / 1 + 1 = **196**. After MaxPooling1D(2): 196 / 2 = **98**. After Conv1D(64, kernel=3): (98 - 3) / 1 + 1 = **96**.

Q23. Why did the 1D CNN (95%) score slightly lower than the Feedforward NN (97.5%)?

A: The CNN's **Embedding layer must be trained from scratch**. With only 198 articles it does not have enough data to learn meaningful word vectors. The TF-IDF approach sidesteps this — it is a simple statistical measure that does not need to be learned. With a larger dataset (tens of thousands of articles), the CNN would likely outperform the Feedforward NN.

7 — Evaluation & Results

Lecture 7

Q24. What evaluation metrics were used? Define each one.

A: **Accuracy** = (TP+TN)/(P+N) — fraction of all correct predictions. **Precision** = TP/(TP+FP) — of all predicted Fake, how many truly are. **Recall** = TP/TP+FN — of all actual Fake articles, how many were caught. **F1-Score** = harmonic mean of Precision and Recall — balances both. Our model: Accuracy=97.5%, Precision(Fake)=1.00, Recall(Fake)=0.95, F1(Fake)=0.97.

Q25. Explain your confusion matrix results for the main NN.

A: TP=19 (Fake correctly caught), TN=20 (Factual correctly passed), FP=0 (no Factual article wrongly flagged as Fake), FN=1 (one Fake article missed). The model has **zero false positives** — it never wrongly accuses a real article of being fake — at the cost of missing 1 fake article.

Q26. What does the confidence distribution plot show?

A: It shows **model calibration**. A well-calibrated model pushes predictions towards 0 (confident Factual) or 1 (confident Fake), not clustering near 0.5 (uncertain). Our model showed strong separation between the two classes, confirming the Sigmoid output is providing meaningful probability estimates.

Q27. Why was validation_split=0.15 used during training?

A: With only 158 training samples, a separate validation set is needed to monitor generalisation without touching the 40-sample test set. 15% of 158 = ~24 samples held out each epoch for validation loss tracking, which drives Early Stopping.

8 — System & Deployment

Q28. How is the trained model served in production?

A: A **FastAPI** backend loads the saved Keras model (nn_model.keras) and the TF-IDF vectorizer (vectorizer.pkl) on startup. The POST /predict endpoint: cleans the input text, converts it to a TF-IDF dense array, calls model.predict(), and returns the label and confidence score as JSON.

Q29. Why must the vectorizer be saved alongside the model?

A: The TF-IDF vectorizer was fitted on the training data — it learned the 3401-word vocabulary and IDF weights. The model was trained on the output of this specific vectorizer. Using a different vectorizer at inference time would produce a different feature vector, making predictions meaningless.

Q30. What would you improve if you had more data?

A: With more data: (1) use a **pre-trained word embedding** (GloVe, Word2Vec) to improve the 1D CNN; (2) try **LSTM/GRU** to capture long-range dependencies in article text; (3) use a **transformer** (BERT) for state-of-the-art results. With the current 198 samples, the Feedforward NN on TF-IDF features is actually the most robust choice.